

Algorithms

Lecture 19:

Minimum Spanning Trees

Dr. Wang, Jingwen

Department of Computer Science

Lecture outline

Announcements/notes

- Daily Problems 15
- Reading ADM page 257-266

Today's lecture

- Daily Problems 14
- Review: BDF and DFS
- Minimum Spanning Trees

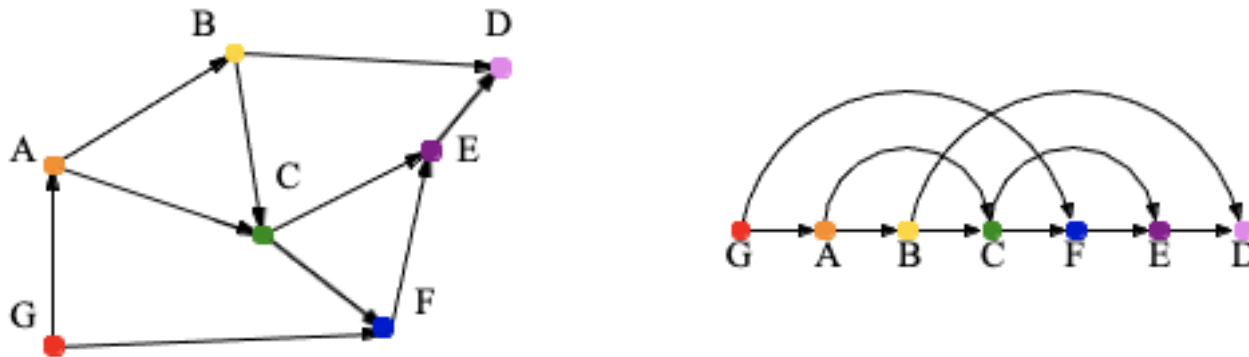
Review: BFS vs. DFS

	Breadth First Search (BFS)	Depth First Search (DFS)
Data Structure	use array and Queue	use array and Stack
Definition	a traversal approach in which we first walk through all nodes on the same level before moving on to the next level.	a traversal approach in which the traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.
Visiting of Siblings/Children	siblings are visited before the children	children are visited before the siblings
Time Complexity	$O(m + n)$ when Adjacency List is used, and $O(n^2)$ when Adjacency Matrix is used, where n stands for # of vertices and m stands for # of edges.	
Applications	find a shortest path in an unweighted graph	acyclic graphs, topological sort

Topological Sorting

Topological Sorting

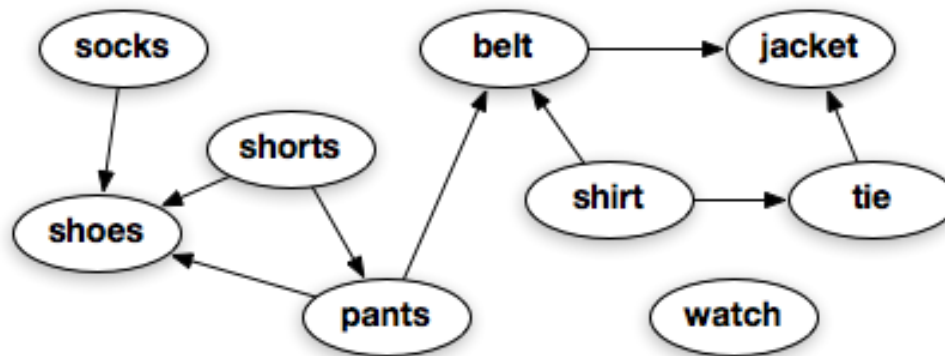
- A directed acyclic graph (DAG) has no cycles.



- A **topological sort** of a graph is an ordering on the vertices so that all edges go from left to right.
- DAGs (and only DAGs) has at least one topological sort** (here G, A, B, C, F, E, D).

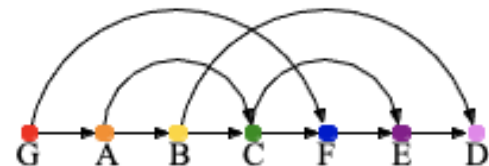
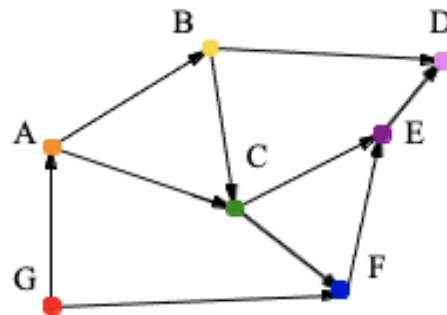
Applications of Topological Sorting

- Topological sorting is often useful in scheduling jobs in their proper sequence. In general, we can use it to order things given precedence constraints.
- Example: Dressing priority schedule



Topological Sorting via DFS

- A directed graph is a DAG if and only if no back edges are encountered during a depth-first search.
- Labeling each of the vertices in the reverse order that they are marked processed in DFS finds a topological sort of a DAG.



Problem of the Day

Problem of the Day

- Your job is to arrange n children in a straight line, facing front. You are given a list of m statements of the form “ i hates j ”. If i hates j , then you do not want put i somewhere behind j , because then i is capable of throwing something at j .
1. Give an algorithm that orders the line, (or says that it is not possible) in $O(m + n)$ time.

2. Suppose instead you want to arrange the children in rows, such that if i hates j then i must be in a lower numbered row than j . Give an algorithm to find the minimum number of rows needed, if it is possible.

Minimum Spanning Trees

Weighted Graph Algorithms

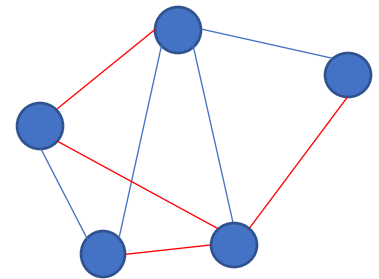
- Beyond DFS/BFS exists an alternate universe of algorithms for edge-weighted graphs.
- Our adjacency list representation quietly supported these graphs:

```
typedef struct edgenode {
    int y;                /* adjacency info */
    int weight;           /* edge weight, if any */
    struct edgenode *next; /* next edge in list */
} edgenode;

typedef struct {
    edgenode *edges[n]; /* adjacency info */
    int degree[n];      /* outdegree of each vertex */
    int nvertices;      /* number of vertices in the graph */
    int nedges;         /* number of edges in the graph */
    int directed;       /* is the graph directed? */
} graph;
```

Spanning Trees

- A tree is a connected graph with no cycles.
- A **spanning tree** is a subgraph of G that covers all the vertices of G and is a tree.
- Discussion:
 - Do the blue edges make a spanning tree?
 - Do the red edges make a spanning tree?
- What graph has the most spanning trees?
- What graph has the fewest spanning trees?
- Give an algorithm to find a spanning tree of a graph.

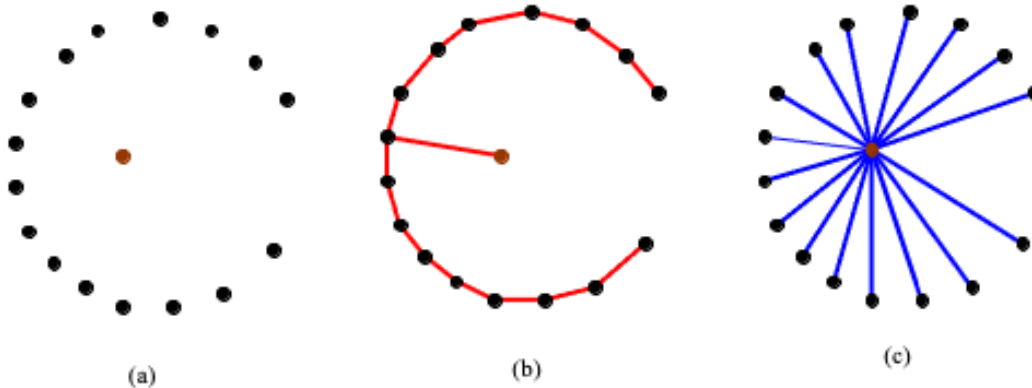


Minimum Spanning Trees

- A **spanning tree** is a subgraph of G that covers all the vertices of G and is a tree.
- A **minimum spanning tree (MST)** of a weighted graph G is the spanning tree of G whose edges sum to minimum weight.
- There can be more than one minimum spanning tree in a graph → consider a graph with identical weight edges.
- Finding an MST is a fundamental problem on weighted graphs.

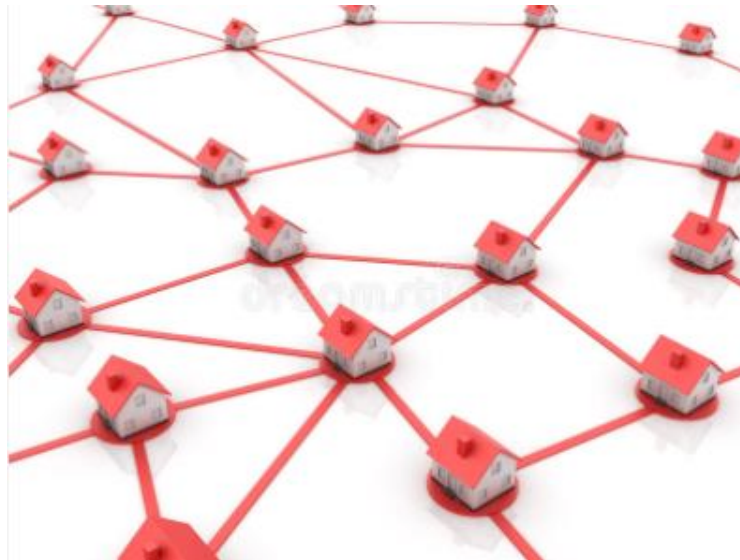
Example of Minimum Spanning Tree

- (a) Edges are not showing on the graph, but every point is connected to every other point, and the weight is the distance between them.
- (b) and (c) are spanning trees of (a)
- Which one of (b) and (c) are the one with less weight.



Applications of Minimum Spanning Trees

- Minimum spanning trees are useful in constructing networks, by describing the way to connect a set of sites using the smallest total amount of wire.



Algorithms to find MSTs

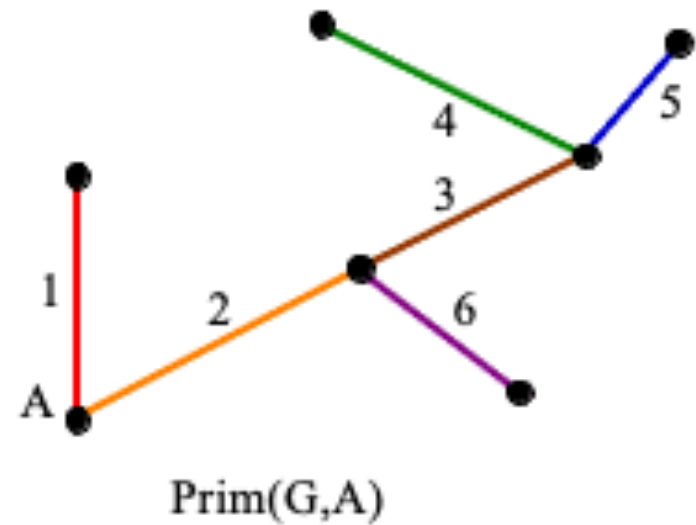
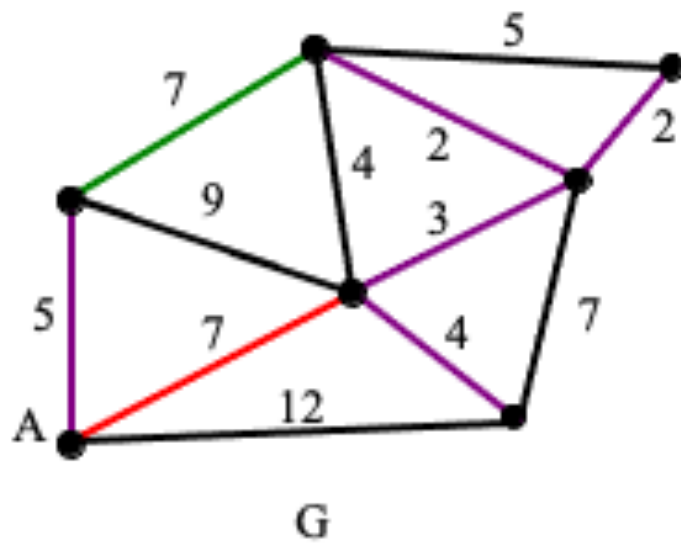
- Prim's Algorithm
- Kruskal's Algorithm

Prim's Algorithm

Prim's Algorithm

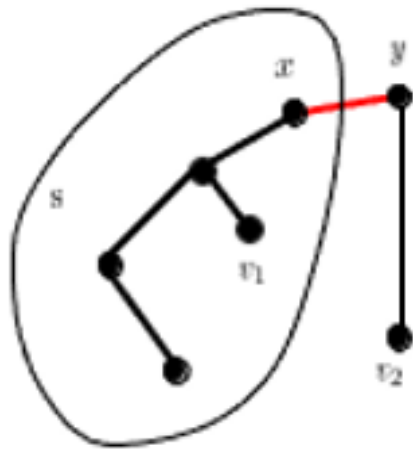
- Prim's algorithm is a greedy algorithm.
- It starts from one vertex and grows the rest of the tree an edge at a time.
- As a greedy algorithm, which edge should we pick?
- The cheapest edge with which can grow the tree by one vertex without creating a cycle.

Prim's Algorithm in Action

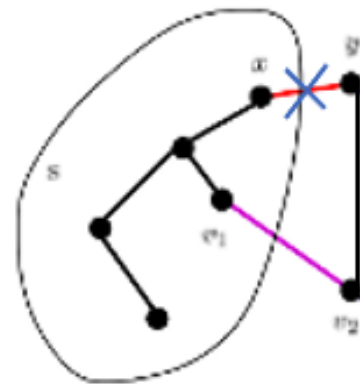


Fringe of an MST

- In the context of a Minimum Spanning Tree (MST), the "**fringe**" is the set of vertices adjacent to the vertices in MST but not yet part of the MST.



(a)



(b)

Prim's Algorithm (Pseudocode)

- During execution, each vertex $\mathbf{v}$ is either
 - in the tree,
 - in the fringe (meaning there exists an edge from a tree vertex to $\mathbf{v}$), or
 - unseen (meaning $\mathbf{v}$ is more than one edge away).

Prim-MST(G)

 Select an arbitrary vertex $\mathbf{s}$ to start with.

 While (there are still non-tree vertices)

 Pick min cost edge between tree/non-tree vertices

 Add the selected edge and vertex to the tree T_{prim} .

- This creates an MST, since no cycle can be introduced and we are choosing the min cost edge each time.

How Fast is Prim's Algorithm?

- That depends on what data structures are used.
- In the simplest implementation, we can simply mark each vertex as tree and non-tree and search always from scratch:

```
Select an arbitrary vertex to start.  
While (there are non-tree vertices)  
    select minimum weight edge between tree and fringe  
    add the selected edge and vertex to the tree
```

- This can be done in $O(nm)$ time, by pick a min-cost edge between tree and fringe, and a total of n iterations.

Weighted Graph Algorithms

- Beyond DFS/BFS exists an alternate universe of algorithms for edge-weighted graphs.
- Our adjacency list representation quietly supported these graphs:

```
typedef struct edgenode {  
    int y;                /* adjacency info */  
    int weight;           /* edge weight, if any */  
    struct edgenode *next; /* next edge in list */  
} edgenode;  
  
typedef struct {  
    edgenode *edges[n]; /* adjacency info */  
    int degree[n];      /* outdegree of each vertex */  
    int nvertices;      /* number of vertices in the graph */  
    int nedges;         /* number of edges in the graph */  
    int directed;       /* is the graph directed? */  
} graph;
```


Prim's Implementation

- We must identify fringe vertices and the minimum cost edge associated with it.

```
int prim(graph *g, int start) {
    int i;           /* counter */
    edgenode *p;     /* temporary pointer */
    bool intree[n];  /* is the vertex in the tree yet? */
    int distance[n]; /* min cost of adding to tree */
    int v;           /* current vertex to process */
    int w;           /* candidate next vertex */

    int dist; /* cheapest cost to enlarge tree */
    int weight = 0; /* tree weight */
    for (i = 1; i <= n; i++) {
        intree[i] = false;
        distance[i] = MAXINT;
        parent[i] = -1;
    }
    distance[start] = 0;
    v = start;
```

n times

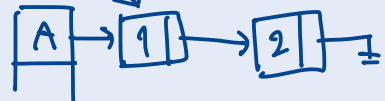
```

while (!intree[v]) {
    intree[v] = true;
    if (v != start) {
        printf("edge (%d,%d) in tree \n", parent[v], v);
        weight = weight + dist;
    }
    p = g->edges[v];
    while (p != NULL) {
        w = p->y;
        if ((distance[w] > p->weight) && (!intree[w])) {
            distance[w] = p->weight;
            parent[w] = v;
        }
        p = p->next;
    }
}

```

O(degree of v)

every node



```

graph LR
    A[A] --> 1[1]
    1 --> 2[2]
    2 --> NULL[ ]

```

```

dist = MAXINT; /* cheapest cost to enlarge tree */
for (i = 1; i <= g->n; i++) {
    if ((!intree[i]) && (dist > distance[i])) {
        dist = distance[i];
        v = i;
    }
}
}

```

*O(n) ↓
of vertices*

```

return (weight);
}

```

Prim's Analysis

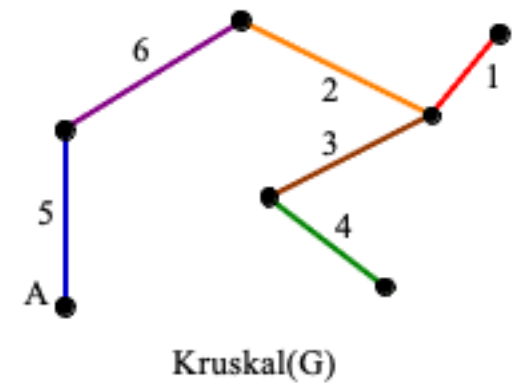
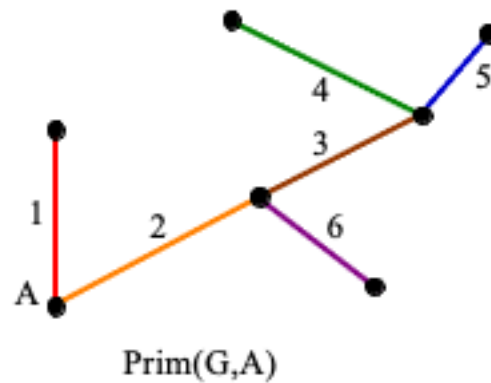
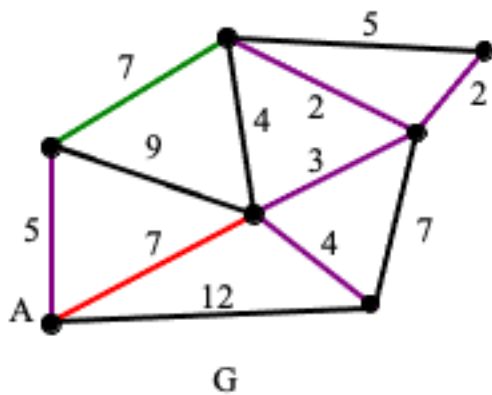
- Finding the minimum weight **fringe-edge** takes $O(n)$, because we iterate through the distance array to find the minimum.
- After adding a vertex v to the tree, by running through its adjacency list in $O(n)$ time we check whether it provides a cheaper way to connect its neighbors to the tree. If so, update the distance value.
- The total time is $n \times O(n) = O(n^2)$.

Kruskal's Algorithm

Kruskal's Algorithm

- Prim's algorithm is optimal on dense graphs.
- Kruskal's algorithm is faster on sparse graphs
- Kruskal's algorithm is also greedy. It repeatedly adds the smallest edge to the spanning tree that does not create a cycle.

Kruskal's Algorithm in Action



How fast is Kruskal's algorithm?

- What is the simplest implementation?
 - Sort the m edges in $O(m \lg m)$ time.
 - For each edge in order, test whether it creates a cycle the forest we have thus far built – if so discard, else add to forest. With a DFS, this can be done in $O(n)$ time (since the MST tree has at most n edges).
- The total time is $O(mn)$

Prim's Algorithm vs. Kruskal's Algorithm

- Both algorithms can find an MST tree of a weighted graph.
- Prim's Algorithm takes either $O(nm)$ or $O(n^2)$ works
- Kruskal's Algorithm takes $O(nm)$ works



▷ *Questions?*